

GELANG

Imperative Language Specification

The language grammar is provided in `grammar.txt` file. Moreover:

1. Arithmetic operations are performed on 64bit integer numbers.
 - Division by zero yield a result of 0 and a remainder of 0 as well.
 - The remainder from division has the same sign as the divisor, just as in Python.
2. Arrays can be declared with integer indices.
 - An identifier `t[i]` refers to the *i*-th element of the array `t`
 - A declaration where the first number is greater than the second is flagged as an error.
 - A declaration of the form `t[-10:10]` means declaring an array with 21 elements indexed from -10 to 10
3. Procedures cannot contain recursive calls. Formal parameters are passed by reference (IN-OUT parameters).
 - Variables used within a procedure must be either formal parameters or declared inside the procedure.
 - The name of the array in formal parameters should be preceded by the letter T.
 - Only previously defined procedures can be called.
 - The formal parameters of the calling procedure and its local variables can be provided as arguments to the called procedures.
 - FOR loop iterators cannot be passed as arguments to a procedure.
4. A FOR loop has a local iterator, which takes values from the value after **FROM** to the value after **TO/DOWNTO** in increments of +1 or -1 accordingly.
 - The number of iterations of the FOR loop is determined at the beginning and cannot change during execution.
 - The iterator of the FOR loop cannot be modified within the loop. Trying to do so will result in compilation error.
5. A REPEAT-UNTIL loop finishes when the condition written after **UNTIL** is satisfied. The loop will execute at least once.
6. The **READ** instruction reads a value from the input and assigns it to a variable, while the **WRITE** instruction outputs the value of a variable or number to the output.
7. The remaining instructions are consistent with their meaning in most programming languages.
8. A **pidentifier** is described by the regular expression `[_a-z]+`.
9. A **num** is an 64bit integer in decimal notation.
10. Lowercase and uppercase letters are distinguished.
11. The program may include comments starting with `#` and valid until the end of the line.

Virtual Machine

The virtual machine consists of a program counter k and a sequence of memory cells p_i , for $i = 0, 1, 2, \dots$ (for technical reasons, $i \leq 2^{62}$). The memory cell p_0 serves as an accumulator. The machine operates on 64bit integer numbers. The machine program consists of a sequence of instructions, which are implicitly numbered starting from zero. At each step, the instruction at index k is executed until a HALT instruction is encountered. Initially, the contents of memory cells are undefined, and the program counter k is set to 0.

Table 1 lists the available instructions along with their interpretations and execution costs. Comments can be included in the program using the format `#comment`. Whitespace in the code is ignored. Jumping to a nonexistent instruction is treated as an error.

Instruction	Interpretation	Time
GET i	Reads a number and stores it in memory cell p_i , then $k \leftarrow k + 1$	100
PUT i	Displays the contents of memory cell p_i , then $k \leftarrow k + 1$	100
LOAD i	$p_0 \leftarrow p_i$, then $k \leftarrow k + 1$	10
STORE i	$p_i \leftarrow p_0$, then $k \leftarrow k + 1$	10
LOADI i	$p_0 \leftarrow p_{p_i}$, then $k \leftarrow k + 1$	20
STOREI i	$p_{p_i} \leftarrow p_0$, then $k \leftarrow k + 1$	20
ADD i	$p_0 \leftarrow p_0 + p_i$, then $k \leftarrow k + 1$	10
SUB i	$p_0 \leftarrow p_0 - p_i$, then $k \leftarrow k + 1$	10
ADDI i	$p_0 \leftarrow p_0 + p_{p_i}$, then $k \leftarrow k + 1$	20
SUBI i	$p_0 \leftarrow p_0 - p_{p_i}$, then $k \leftarrow k + 1$	20
SET x	$p_0 \leftarrow x$, then $k \leftarrow k + 1$	50
HALF	$p_0 \leftarrow \lfloor p_0/2 \rfloor$, then $k \leftarrow k + 1$	5
JUMP j	$k \leftarrow k + j$	1
JPOS j	If $p_0 > 0$, then $k \leftarrow k + j$, otherwise $k \leftarrow k + 1$	1
JZERO j	If $p_0 = 0$, then $k \leftarrow k + j$, otherwise $k \leftarrow k + 1$	1
JNEG j	If $p_0 < 0$, then $k \leftarrow k + j$, otherwise $k \leftarrow k + 1$	1
RTRN i	$k \leftarrow p_i$	10
HALT	Stops the program	0

Table 1: Instruction set of the virtual machine